

Capítulo 9

Integraciones externas: servicios de terceros, pagos e IA

Desarrollo de Software IX · Código 1493 · Módulo III

Propósito del capítulo

Desarrollar la teoría de la integración del backend con servicios de terceros: el contrato con APIs externas, las pasarelas de pago en línea y el consumo de modelos de inteligencia artificial, con sus riesgos, costos y buenas prácticas.

Tabla de contenido

1. Panorama del capítulo	3
2. El contrato con una API de terceros	3
3. Pasarelas de pago	5
4. Consumo de modelos de IA, ML y DL	7
5. La capa de integración en la arquitectura	8
6. Caso guía: fichas compradas y asistente del juego	9
7. Actividades sugeridas	9
8. Rúbrica breve	9
9. Cierre	10

1. Panorama del capítulo

Ninguna aplicación moderna se construye sola. Detrás de una compra en línea hay una pasarela de pago; detrás de un asistente conversacional hay un modelo de IA servido por un proveedor; detrás de un correo de confirmación, un mapa o una notificación hay servicios de terceros. El programa del curso ubica estos temas en el diseño del backend (Módulo I, Unidad III): consumo de servicios externos, pasarela de pago y consumo de modelos de IA, ML o DL. Este capítulo los desarrolla con la profundidad de implementación del Módulo III.

La idea que organiza todo el capítulo es la de FRONTERA. Cuando el backend llama a un servicio externo, cruza una frontera de confianza y de control: el código del otro lado no es nuestro, puede fallar, puede tardar, puede cambiar y casi siempre cuesta dinero. Integrar bien no es solo lograr que la llamada funcione una vez; es diseñar para que el sistema siga siendo correcto cuando el tercero se comporte mal.

Idea central

Un servicio externo es capacidad alquilada: aporta en horas lo que tomaría meses construir (cobrar tarjetas, inferencia de IA, correo, mapas), a cambio de dependencia. La ingeniería está en obtener la capacidad sin heredar la fragilidad.

2. El contrato con una API de terceros

2.1 Qué se firma al integrar

Toda integración se apoya en un contrato técnico: la documentación de la API define recursos, formatos, códigos de error, versiones y límites de uso. A diferencia de una API propia, este contrato no se puede modificar: se acepta y se monitorea. Tres cláusulas merecen atención especial antes de escribir la primera línea de código:

- Versionado: los proveedores publican versiones (v1, v2, encabezados de versión) y deprecian las antiguas. Fijar la versión explícitamente evita que un cambio del proveedor rompa producción.
- Límites de uso (rate limits): cuántas solicitudes por segundo o por día acepta el servicio. Excederlos produce respuestas 429 y, en servicios pagos, costos inesperados.
- Acuerdos de disponibilidad: ningún tercero promete 100%. El diseño debe asumir que el servicio estará caído alguna vez — la pregunta es qué hace nuestra aplicación ese día.

2.2 Autenticación entre servicios: las llaves

Las APIs comerciales identifican a quien las consume mediante credenciales: típicamente una API key (una cadena secreta asociada a la cuenta) o un flujo OAuth cuando se actúa en nombre de un usuario. La regla de oro es que las credenciales viven EXCLUSIVAMENTE en el servidor: en variables de entorno, nunca en el código fuente, nunca en el repositorio y nunca en el frontend, porque todo lo que llega al navegador es público.

```
# .env (fuera del repositorio, listado en .gitignore)
STRIPE_SECRET_KEY=sk_test_...
OPENAI_API_KEY=sk-proj-...

# El backend las lee del entorno
const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY)
```

Una llave expuesta es un incidente

Una API key subida a un repositorio público se considera comprometida aunque se borre después: los recolectores automáticos escanean GitHub en segundos. El procedimiento correcto es revocarla y emitir una nueva, no eliminar el commit. Por eso los proveedores distinguen llaves de prueba (sk_test_) y de producción (sk_live_).

2.3 Resiliencia: diseñar para el fallo ajeno

Una llamada a un tercero es una operación de red: puede tardar, perderse o devolver error. El backend debe decidir de antemano su comportamiento ante cada caso, en lugar de heredar el fallo y propagarlo al usuario. Cuatro mecanismos cubren la mayoría de las situaciones:

Mecanismo	Qué problema ataca	Idea de implementación
Timeout	El tercero no responde y la petición queda colgada ocupando recursos.	Toda llamada externa lleva un tiempo máximo de espera; al vencerse, se trata como error.
Reintento con espera creciente	Fallos transitorios (red, 503) que se resuelven solos.	Reintentar 2-3 veces con pausas crecientes (backoff); solo en operaciones seguras de repetir.
Idempotencia	Repetir una operación con efectos (un cobro) podría duplicarla.	Enviar una clave de idempotencia: el proveedor ejecuta una sola vez aunque reciba la petición dos veces.
Degradación controlada	El servicio está caído y no hay reintento que valga.	Responder con un mensaje claro o una alternativa limitada, en lugar de un error críptico.

La idempotencia merece subrayarse porque conecta con los pagos: si el backend cobra y la respuesta se pierde en la red, ¿se atreve a reintentar? Sin clave de idempotencia, reintentar arriesga un doble cobro; con ella, el reintento es seguro porque el proveedor reconoce la operación ya ejecutada.

2.4 Webhooks: cuando el tercero llama

El modelo petición-respuesta no alcanza para procesos que tardan o que ocurren del lado del proveedor: la confirmación definitiva de un pago, un reembolso, una disputa. Para eso existen los webhooks: la aplicación registra una URL propia y el proveedor le envía una petición HTTP cada vez que ocurre un evento. El flujo se invierte — nuestro backend pasa a ser servidor del tercero.

Aspecto	Consultar periódicamente (polling)	Webhook (push)
Quién inicia	Nuestra aplicación pregunta una y otra vez.	El proveedor avisa cuando hay novedad.
Latencia	Depende de la frecuencia de consulta.	Cercana al evento real.

Aspecto	Consultar periódicamente (polling)	Webhook (push)
Costo	Solicitudes constantes, casi todas vacías.	Solo tráfico útil.
Complejidad	Baja: es un GET repetido.	Media: endpoint público, verificación de firma, reintentos del proveedor.

Como el endpoint de webhook es público, cualquiera podría enviarle peticiones falsas. Por eso los proveedores firman cada evento (un encabezado con HMAC sobre el cuerpo) y la aplicación debe verificar la firma antes de confiar en el contenido — el mismo principio de firma que sustenta el JWT del Capítulo 5.

3. Pasarelas de pago

3.1 Por qué nadie procesa tarjetas directamente

Recibir un número de tarjeta impone obligaciones técnicas y legales severas: el estándar PCI-DSS regula cómo se almacena, transmite y procesa ese dato, y una filtración compromete dinero real de los usuarios. La industria resolvió el problema con la pasarela de pago (Stripe, PayPal, y procesadores locales): un tercero certificado que recibe el dato sensible directamente del navegador del cliente, de modo que el número de tarjeta NUNCA toca nuestro servidor.

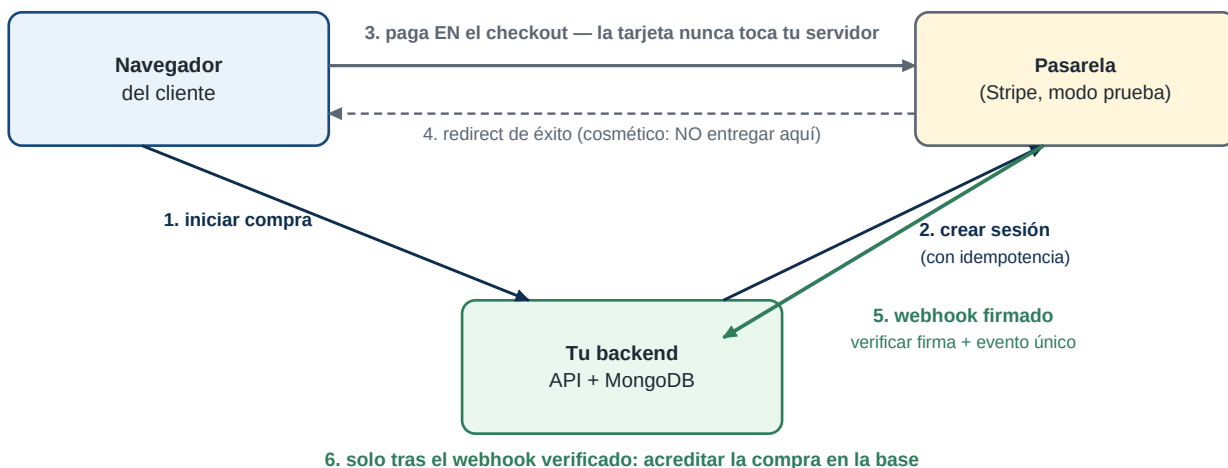
El mecanismo que lo hace posible es la tokenización: el frontend entrega los datos de la tarjeta a la pasarela (mediante su formulario embebido o su página de checkout) y recibe a cambio un token — una referencia opaca y de un solo uso. Nuestro backend opera con tokens e identificadores, jamás con tarjetas.

3.2 El flujo de un cobro

- 1 El usuario decide comprar; el frontend pide al backend iniciar el cobro.
- 2 El backend, con su llave secreta, crea en la pasarela una intención de pago por un monto y moneda (en Stripe: Checkout Session o PaymentIntent).
- 3 La pasarela devuelve una URL de checkout o un secreto de cliente; el backend lo entrega al frontend.
- 4 El usuario completa el pago EN la interfaz de la pasarela (página alojada o formulario embebido); la tarjeta viaja del navegador a la pasarela, sin pasar por nuestro servidor.
- 5 La pasarela redirige al usuario a una página de éxito o fallo — pero esa redirección es solo cosmética.
- 6 La confirmación REAL llega por webhook: la pasarela notifica el evento de pago completado y solo entonces el backend entrega el producto (acredita fichas, habilita el tablero, marca la orden pagada).

```
// Crear la sesión de cobro (backend)
const session = await stripe.checkout.sessions.create({
  mode: 'payment',
  line_items: [{
    price_data: {
      currency: 'usd',
      product_data: { name: 'Paquete de 100 fichas' },
      unit_amount: 500, // centavos: $5.00
    },
    quantity: 1,
  }],
  success_url: 'https://miapp.com/compra/exito',
  cancel_url: 'https://miapp.com/compra/cancelado',
}, { idempotencyKey: orderId }) // reintento seguro

return c.json({ url: session.url })
```



3.3 La verdad llega por webhook

El error de diseño más común en pagos es entregar el producto en la página de éxito. Esa página puede no cargarse (el usuario cierra el navegador), o puede forjarse (alguien visita la URL sin pagar). La fuente de verdad es el evento firmado que la pasarela envía al webhook; la entrega del producto debe depender de él y de nada más. Además, como el proveedor reenvía eventos si no recibe respuesta, el manejador debe ser idempotente: procesar dos veces el mismo evento no puede acreditar fichas dos veces.

```
// Webhook de pagos (backend)
app.post('/webhooks/stripe', async (c) => {
  const firma = c.req.header('stripe-signature')
  const evento = stripe.webhooks.constructEvent(
    await c.req.text(), firma, process.env.STRIPE_WEBHOOK_SECRET
  ) // lanza error si la firma no es válida

  if (evento.type === 'checkout.session.completed') {
    const yaProcesado = await Orden.findOne({ eventoId: evento.id })
    if (!yaProcesado) await acreditarCompra(evento.data.object)
  }
  return c.json({ received: true })
})
```

3.4 El modo de prueba

Toda pasarela sería ofrece un modo de prueba completo: llaves de prueba, tarjetas ficticias (la clásica 4242 4242 4242 4242), y un panel donde se ven los cobros simulados. El ciclo de desarrollo ocurre íntegramente en modo prueba; pasar a producción es cambiar las llaves por las reales y verificar de nuevo el flujo. Mezclar llaves de prueba y producción es un error de configuración clásico — otra razón para que las llaves vivan en variables de entorno por ambiente.

Conexión con los proyectos del curso

Tanto el MVP del Parcial #2 como el proyecto del juego de damas integran una pasarela en modo prueba. El criterio de calidad es el de este capítulo: tarjeta que nunca toca el backend, entrega del producto condicionada al webhook verificado, y manejo idempotente de eventos.

4. Consumo de modelos de IA, ML y DL

4.1 Modelos como servicios

Entrenar y servir un modelo de lenguaje o de visión exige hardware y equipos especializados; por eso la forma dominante de incorporar IA a una aplicación es consumirla como servicio: proveedores como OpenAI, Anthropic o Google exponen sus modelos detrás de APIs HTTP, y agregadores como OpenRouter unifican varios proveedores tras una sola interfaz. Para el backend, un modelo es entonces un servicio externo más — aplican TODAS las reglas de la sección 2: llaves en el servidor, timeouts, manejo de errores y atención al costo — con particularidades propias.

4.2 Anatomía de una inferencia

En los modelos de lenguaje, la petición típica envía una conversación (lista de mensajes con roles: system para las instrucciones, user para la entrada) y parámetros como el modelo elegido y la temperatura (cuánta variabilidad se permite en la salida). La respuesta llega medida en tokens — fragmentos de texto de unos pocos caracteres — que son también la unidad de cobro: se paga por tokens de entrada y de salida.

```
// Consumo de un modelo desde el backend
const respuesta = await fetch('https://api.openai.com/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${process.env.OPENAI_API_KEY}`,
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    model: 'gpt-5.2',
    messages: [
      { role: 'system', content: 'Eres un asistente del campus. Responde en una oración.' },
      { role: 'user', content: pregunta },
    ],
  }),
  signal: AbortSignal.timeout(20000), // timeout explícito
})
const data = await respuesta.json()
return c.json({ texto: data.choices[0].message.content })
```

Las inferencias tardan segundos, no milisegundos. Para que la espera no degrade la experiencia, los proveedores ofrecen streaming: la respuesta llega token por token (mediante eventos enviados por el servidor) y la interfaz la muestra a medida que se genera — el patrón que el usuario reconoce de cualquier

asistente moderno. Del lado del frontend, esto conecta con los estados de carga del Capítulo 8 y con la interfaz optimista de React 19.

4.3 Costos, límites y caché

- El costo es por uso y crece con el tráfico: una funcionalidad de IA sin límites puede generar una factura inesperada. Los proveedores permiten fijar presupuestos y alertas; la aplicación debe además limitar usos por usuario.
- Los tokens de entrada también cuestan: enviar contexto innecesario (historiales completos, documentos enteros) multiplica el gasto sin mejorar la respuesta.
- Respuestas repetidas se cachean: si cien usuarios hacen la misma pregunta, no hacen falta cien inferencias.
- La elección del modelo es una decisión de viabilidad técnica: los modelos grandes razonan mejor y cuestan más; los pequeños bastan para clasificar, resumir o autocompletar. Elegir el más capaz 'por si acaso' es pagar capacidad que no se usa.

4.4 Diseño responsable

Consumir IA introduce riesgos que no existen con otros servicios. La salida del modelo es texto generado, no verdad verificada: puede contener errores con apariencia de certeza, y si se muestra sin filtro o se ejecuta como código, el riesgo es de la aplicación, no del proveedor. Además, lo que se envía en el prompt sale de la organización: datos personales o sensibles requieren revisar las políticas de retención del proveedor. Y como toda entrada de usuario, el texto que se interpola en un prompt puede intentar manipular las instrucciones (inyección de prompt) — el prompt del sistema y la validación de salidas son la defensa.

Criterio	Pregunta de diseño
Capacidad	¿La tarea exige razonamiento complejo o basta un modelo pequeño y rápido?
Costo	¿Cuánto cuesta el caso de uso por usuario al mes? ¿Hay límites y alertas configurados?
Latencia	¿La experiencia tolera segundos de espera? ¿Hace falta streaming?
Privacidad	¿Qué datos viajan en el prompt y qué hace el proveedor con ellos?
Reemplazabilidad	¿Qué tan difícil sería cambiar de proveedor o de modelo el próximo semestre?

5. La capa de integración en la arquitectura

Si las llamadas a terceros se esparcen por controladores y componentes, cada cambio de proveedor o de API se convierte en una cirugía sobre toda la aplicación. La práctica correcta es aislar cada integración en un módulo propio — un adaptador — que expone al resto del sistema una interfaz en términos del dominio (cobrarOrden, generarResumen) y esconde los detalles del proveedor (endpoints, formatos, llaves).

```
// services/pagos.js – el resto de la app no sabe que es Stripe
export async function crearCobro(orden) { /* Stripe aquí */ }
export async function verificarEvento(req) { /* firma aquí */ }
```

```
// services/ia.js – el resto de la app no sabe qué modelo es
export async function resumir(texto) { /* proveedor aquí */ }
```


- Probar se vuelve posible: en las pruebas, el adaptador se sustituye por una versión simulada y la lógica propia se verifica sin tocar el servicio real (ni pagar por él).
- Cambiar de proveedor se vuelve local: solo se reescribe el adaptador, no la aplicación.
- La configuración por ambiente (llaves de prueba en desarrollo, reales en producción) vive en un solo lugar.

Principio de diseño

La aplicación habla su propio idioma; los adaptadores traducen. Si el nombre de un proveedor aparece en un componente de React, la frontera está rota.

6. Caso guía: fichas compradas y asistente del juego

Considere una aplicación de juego en línea que vende paquetes de fichas y ofrece un asistente que explica las reglas. El flujo de compra: el usuario elige un paquete, el backend crea la sesión de cobro con clave de idempotencia, el usuario paga en el checkout de la pasarela con una tarjeta de prueba, y las fichas se acreditan únicamente cuando llega el webhook con firma válida — el manejador descarta eventos repetidos. El asistente: el frontend envía la pregunta al backend, que arma el prompt con instrucciones fijas del sistema, llama al modelo con timeout, limita cada usuario a un número de consultas y muestra la respuesta en streaming con sus estados de carga y error.

Este caso obliga a usar todo el capítulo: llaves por entorno, dos adaptadores (pagos e IA), un webhook público con verificación de firma, idempotencia de eventos, control de costos de inferencia y degradación honesta cuando alguno de los dos terceros falle.

7. Actividades sugeridas

- Configurar una pasarela en modo prueba y completar un cobro con la tarjeta 4242 4242 4242 4242, identificando en qué momento exacto del flujo conviene entregar el producto.
- Provocar un evento de webhook repetido (reenvío desde el panel del proveedor) y verificar que la aplicación no acredita dos veces.
- Consumir un modelo de IA desde el backend con timeout y límite de usos por usuario; medir tokens y costo de diez consultas reales.
- Refactorizar una integración existente hacia un adaptador y sustituirlo por una versión simulada en una prueba.
- Redactar media página de viabilidad técnica comparando dos proveedores (de pagos o de IA) con los criterios del capítulo.

8. Rúbrica breve

Criterio	Excelente	Satisfactorio	Insuficiente
Manejo de secretos	Llaves por ambiente, fuera del repo y solo en el servidor.	Llaves en el servidor con descuidos menores.	Llaves en el código, el repo o el frontend.

Criterio	Excelente	Satisfactorio	Insuficiente
Flujo de pago	Tokenización, entrega condicionada al webhook verificado e idempotente.	Cobro funcional con verificación parcial.	Entrega en la página de éxito o sin verificación.
Consumo de IA	Llamada desde el backend con timeout, límites, control de costo y validación de salida.	Funciona con controles parciales.	Llave o llamada expuesta en el frontend, sin límites.
Arquitectura	Integraciones aisladas en adaptadores probables y reemplazables.	Aislamiento parcial.	Llamadas a terceros esparcidas por la aplicación.

9. Cierre

Integrar terceros es la habilidad que convierte un ejercicio académico en un producto: casi todo lo valioso — cobrar, inferir, notificar — pasa por una frontera con un servicio ajeno. La disciplina es siempre la misma: contrato leído, secretos protegidos, fallos previstos, verdad verificada por firma y dependencias aisladas detrás de adaptadores. Con este capítulo queda cubierto el último tramo del diseño de backend que plantea el programa del curso: no solo construir servicios propios, sino componerlos responsablemente con los del resto del mundo.